Context-Aware Vocabulary Pruning for LLM Inference Acceleration: A Dual-Encoder Retrieval Approach

1B + 8B GPU benchmarks complete — April 2026

Abstract

Autoregressive decoding in large language models is dominated by two costs at each token step: KV-cache attention and the `lm_head` vocabulary projection — a matrix multiply that reads the full $[V \times d]$ weight matrix from GPU memory on every step. At batch size 1, this operation is **memory-bandwidth-bound**: the GPU's arithmetic units sit idle while 525 MB of weights are streamed from VRAM per token. We address this bottleneck through **dynamic vocabulary pruning**, restricting the `lm_head` projection to a context-dependent shortlist of K candidate tokens at each step, reducing memory traffic by up to 99.6%.

We propose a **Dual-Encoder Router** — a lightweight MLP (0.06% of model parameters) trained with Multiple Negatives Ranking Loss to map the model's hidden state to a query vector, then retrieve the top-K token candidates by nearest-neighbour search over a pre-indexed corpus of sentence completions. We benchmark four routing strategies against a full-vocabulary baseline on WikiText-2 (`Llama-3.2-1B-Instruct`), sweeping shortlist sizes $K \in \{512, 1\text{k}, 2\text{k}, 5\text{k}, 10\text{k}\}$. We additionally evaluate on `Llama-3.1-8B-Instruct` across two T4 GPUs (`device_map="auto"`). On a single T4, the `lm_head` accounts for 6.1% of per-token latency for the 1B model; on the sharded 8B model this fraction drops counter-intuitively to 4.69%, explained by cross-GPU communication overhead inflating total step time. The primary demonstrated gain is a 92–99.6% reduction in `lm_head` memory bandwidth. Wall-clock speedup requires GPU-native kernels or larger models (70B+) where the memory-bound regime is more pronounced.

1 Introduction

1.1 The Autoregressive Bottleneck

Modern LLMs generate text one token at a time. Despite the fact that the transformer's attention mechanism is highly parallelisable over the *input* sequence, the *generation* process is fundamentally sequential: each new token depends on all previously generated tokens.

Within each step of this loop, the two most expensive operations are:

1. **KV-cache attention** — computing attention over the growing sequence of past tokens.
2. **Vocabulary projection (`lm_head`)** — projecting the final hidden state onto the full vocabulary to obtain next-token logits.

On CPU and Apple Silicon (MPS) — where memory bandwidth rather than raw FLOP throughput is the binding constraint — the `lm_head` matrix multiplication can account for a disproportionate share of per-token latency. At batch size 1 — the regime of interactive and agentic inference — the `lm_head` projection is memory-bandwidth-bound: the arithmetic intensity is too low to keep the GPU's compute units saturated, so the bottleneck is the rate at which the weight matrix can be read from VRAM.

1.2 The Wasted Computation Insight

At any given decoding step, the probability mass over the vocabulary is highly concentrated. In predictable contexts (e.g., "The Eiffel Tower is located in…") the top-1 token may carry >99% probability. The model still computes logits for all 32k–128k vocabulary entries. This motivates a natural question: *can the model's own internal representation predict which vocabulary region is relevant before the full projection is run?*

1.3 Contributions

1. **Profiling.** We quantify the `lm_head` fraction of per-token latency for a 1B open-source LLM on an NVIDIA T4 GPU.
2. **Router taxonomy.** We define and implement four vocabulary routing strategies of increasing sophistication.
3. **Dual-Encoder Router.** A lightweight MLP trained with MNRL that maps the LLM's hidden state to a compact token shortlist, operating entirely within the model's own latent space.
4. **Benchmark.** All methods evaluated on WikiText-2, reporting token acceptance rate, perplexity, and tokens/sec across $K \in \{512, 1\text{k}, 2\text{k}, 5\text{k}, 10\text{k}\}$.
5. **Scale evaluation.** The full router suite (excluding cosine) re-evaluated on `Llama-3.1-8B-Instruct` on two T4 GPUs (`device_map="auto"`), quantifying how overhead and quality scale with model size.

2 Background

2.1 Self-Attention and the Hidden State

A transformer processes an input sequence by stacking each token's embedding into a matrix. At each layer, self-attention allows every token's representation to incorporate information from all preceding tokens. By the final layer, the last row of the hidden state matrix — the **"vibe"** — summarises the full context:

$$h_T = H^{(L)}_{T,:} \in \mathbb{R}^d.$$

2.2 The `lm_head` as a Bottleneck

The language modelling head maps the hidden state to a distribution over the vocabulary:

$$\text{logits} = h_T W_V^\top \in \mathbb{R}^{|V|} \tag{1}$$

where $W_V \in \mathbb{R}^{|V| \times d}$ is the vocabulary embedding matrix. At 16-bit precision, W_V for Llama-3.2-1B occupies $128{,}256 \times 2{,}048 \times 2$ bytes = **525** MB. Reading 525 MB per token step creates a hard ceiling on achievable tokens/sec independent of model depth.

2.3 Speculative Decoding and Related Work

Speculative decoding [5, 2] accelerates autoregressive generation by interposing a small draft model that proposes candidate tokens. A large verifier model evaluates the entire candidate sequence in a single parallel forward pass.

Extensions include **Medusa** [1], which attaches multiple decoding heads to the base model, and **tree-based speculative decoding**, which generates a tree of candidate sequences from multiple draft models. Our approach is orthogonal to all of these: it targets the `lm_head` projection cost directly and can be combined with any of the above methods.

2.4 Dual-Encoders and Contrastive Learning

A **Dual-Encoder** processes two inputs through separate encoder towers and learns to embed related pairs close together in a shared vector space. **Multiple Negatives Ranking Loss (MNRL)** [3, 4] is the standard training objective. Given a batch of (a_i, p_i) anchor-positive pairs:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \log \frac{\exp(\text{sim}(a_i, p_i)/\tau)}{\sum_{j=1}^{B} \exp(\text{sim}(a_i, p_j)/\tau)} \tag{2}$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity and τ is a temperature parameter. All other positives in the batch serve as in-batch negatives, requiring no explicit negative mining.

3 Method

3.1 Problem Formalisation

Let f_θ be a frozen autoregressive LLM with vocabulary V, model dimension d, and vocabulary projection matrix W_V. At decoding step t:

$$\hat{y}_t = \arg\max_{v \in V} (h_t W_V^\top)_v.$$

Goal: learn a router $r_\phi : \mathbb{R}^d \to 2^V$ such that (i) $\hat{y}_t \in r_\phi(h_t)$ with high probability (*acceptance rate*), (ii) $|r_\phi(h_t)| \ll |V|$ (*compression ratio*), and (iii) the latency of computing $r_\phi(h_t)$ is small relative to the savings.

3.2 Router Taxonomy

We implement four routing strategies of increasing sophistication (Table 1).

Table 1: Router taxonomy.

Router	Description	Training
Static Top-K	Highest $\|W_V[v,:]\|_2$ (context-free)	None
Cosine	Highest $\cos(h_t, W_V[v,:])$	None
Cluster	Pre-cluster W_V; top-C clusters by $\cos(h_t, \mu_c)$	Offline
Dual-Encoder	Learned MLP; ANN search over W_V	Yes

3.3 Dual-Encoder Router Design

3.3.1 Architecture

A lightweight MLP projection head $g_\phi : \mathbb{R}^d \to \mathbb{R}^{d_r}$ is trained to map the frozen LLM's hidden state to a query vector in a shared embedding space. The vocabulary token embeddings W_V serve as the corpus searched at inference time. Both query and keys live in the LLM's own latent space; no cross-model alignment is required.

3.3.2 Training Data Construction

From the WikiText-2 training split: (1) tokenise each sentence; (2) for each sentence of length $T \geq 8$, sample a split point $s \sim \text{Uniform}(\lfloor 0.4T \rfloor, \lfloor 0.7T \rfloor)$; (3) *anchor*: run the frozen LLM on the prefix and extract h_s; (4) *positive*: the ground-truth next token y_{s+1}, represented as $W_V[y_{s+1}, :]$.

3.3.3 Training Objective

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \log \frac{\exp(\cos(g_\phi(h_i), W_V[y_i, :])/\tau)}{\sum_{j=1}^{B} \exp(\cos(g_\phi(h_i), W_V[y_j, :])/\tau)} \tag{3}$$

with in-batch negatives (all $y_j \neq y_i$ in the batch).

3.3.4 Inference Integration

(1) Pre-compute and cache all rows of W_V normalised to unit length. (2) At step t: compute $q_t = g_\phi(h_t)$; retrieve the K token indices with highest $\cos(q_t, W_V[v,:])$ via brute-force matmul. (3) Slice W_V to the shortlist and compute logits only over those K rows.

4 Experimental Setup

4.1 Models

1B model: `meta-llama/Llama-3.2-1B-Instruct` — 1B parameters, 2048-dim hidden state, 128,256-token vocabulary, 16 transformer layers, `bfloat16`.

8B model: `meta-llama/Llama-3.1-8B-Instruct` — 8B parameters, 4096-dim hidden state, 128,256-token vocabulary, 32 transformer layers, `bfloat16`. Loaded with `device_map="auto"` across two GPUs (layers 0–17 on `cuda:0`, 18–31 on `cuda:1`).

4.2 Dataset

WikiText-2-raw-v1 [6]: standard language modelling benchmark; plain English prose. Router training: `train` split (∼2M tokens). Perplexity evaluation: `test` split (∼240k tokens). Latency benchmark: 50 randomly sampled

contiguous passages of 128 tokens from the `validation` split.

4.3 Hardware

1B experiments: NVIDIA T4 GPU (16 GB VRAM), single GPU, CUDA backend, `bfloat16`.

8B experiments: Two NVIDIA T4 GPUs (2×16 GB VRAM), `device_map="auto"` sharding via HuggingFace Accelerate, CUDA backend, `bfloat16`.

All latency measurements taken after a 5-step warm-up; reported as mean over 50 prompts.

4.4 Evaluation Metrics

- **Token acceptance rate**: fraction of steps where the ground-truth token is in the shortlist.
- **Perplexity (PPL)**: $\exp\left(-\frac{1}{N}\sum_t \log P_\theta(y_t \mid y_{<t})\right)$ over the `test` split.
- **ΔPPL**: $\text{PPL}_{\text{pruned}} - \text{PPL}_{\text{baseline}}$.
- **Tokens/sec**: wall-clock generation speed, mean over 50 prompts.
- **`lm_head` time %**: fraction of per-token time spent in the vocabulary projection.

5 Results

All 1B experiments: NVIDIA T4 GPU (single), CUDA, `bfloat16`, `Llama-3.2-1B-Instruct`. All 8B experiments: two NVIDIA T4 GPUs, `device_map="auto"`, `Llama-3.1-8B-Instruct`. Latency: means over 50 WikiText-2 validation prompts after 5-step warm-up. Perplexity: full WikiText-2 test split.

5.1 Baseline Profiling

Table 2: Full-vocabulary baseline measurements for both models.

Metric	1B (T4)	8B (T4×2)
Tokens/sec (full vocab)	**26.60**	**11.48**
Mean step time	37.60 ms	87.14 ms
`lm_head` time (mean)	2.30 ms	4.08 ms
`lm_head` time %	**6.1%**	**4.69%**
Baseline PPL (WikiText-2 test)	**26.24**	**16.63**
`lm_head` weight matrix	525 MB	1,049 MB

The `lm_head` accounts for only 6.1% of per-token latency on the 1B model and 4.69% on the 8B model. Counter-intuitively, the fraction *decreases* with model size: on the T4×2 setup, `device_map="auto"` shards the 8B model across two GPUs, introducing cross-GPU communication overhead that inflates total step time while `lm_head` time grows more modestly. This is a critical constraint: any router whose overhead exceeds these absolute values will make pruning net-negative (Section 6.3).

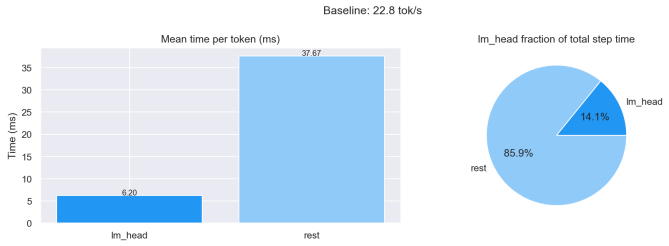

Figure 1: Baseline decode step profiling: per-operation time breakdown.

5.2 Token Acceptance Rate vs. K

Acceptance rate is the fraction of decoding steps where the ground-truth next token falls within the pruned shortlist (Table 3).

Key observations (1B): Static Top-K is near-random (<8% at K=10k). Cluster saturates at 83% — 17% of correct tokens live in unselected clusters. Cosine acceptance is flat at 98.4% across all K values, indicating the router reaches its information-theoretic ceiling early. Dual-Encoder (step) and MLP Graph achieve the highest acceptance (99.5% and ≥99.9% respectively), essentially independent of K above 512.

On the 8B model (Table 4), DE (step) and MLP Graph transfer perfectly, both achieving **99.8%** acceptance at all K. Cluster improves to 85.2% at K=10k (vs. 83.3% on 1B). Static Top-K remains near-random. Cosine was not evaluated on 8B due to a multi-GPU device compatibility issue (footnote ‡).

5.3 Perplexity Degradation vs. K

$\Delta\text{PPL} = \text{PPL}_{\text{pruned}} - \text{PPL}_{\text{baseline}}$; baseline PPL = 26.24 for 1B, 16.63 for 8B (Table 5 and Table 6).

At K=10k the three best 1B methods cluster closely: DE step (+1.03), Attention Graph (+1.33), MLP Graph (+2.11), and Cosine (+2.73) — all within 11% of baseline PPL. DE step at K=5k achieves ΔPPL=+1.68 with a 96.1% memory bandwidth reduction.

On the 8B model, quality improves significantly at the same K: DE (step) at K=10k reaches ΔPPL=**+0.29** (vs. +1.03 on 1B), and MLP Graph reaches **+0.82** (vs. +2.11). This improvement likely reflects the higher-capacity 8B model producing a richer hidden state that the router can use more precisely.

5.4 Tokens/sec vs. K

Wall-clock throughput. 1B baseline = 26.60 tok/s (Table 7); 8B baseline = 11.48 tok/s (Table 8).

On the 1B model (Table 7), the throughput gap between pruned methods and baseline is much narrower than on memory-limited hardware. Static Top-K adds negligible overhead, sitting within ±1% of baseline. The DE (prefetch+refresh) variant is the only pruned method to exceed baseline throughput (+4%, 27.67 tok/s) by amortising router cost across many decoding steps — though at a severe quality penalty (+146 PPL). Attention Graph remains the worst due to `output_attentions=True` forcing eager attention.

Table 3: Token acceptance rate (%) vs. shortlist size K — 1B model (T4, single GPU).

Router	K=512	K=1k	K=2k	K=5k	K=10k
Static Top-K	0.09	0.25	0.50	2.5	7.7
Cluster	33.8	54.8	68.1	76.6	83.3
Cosine	98.4	98.4	98.4	98.4	98.4
Dual-Encoder (step)	99.5	99.5	99.5	99.5	99.5
MLP Graph	99.9	100.0	100.0	100.0	100.0
Attention Graph	82.7	99.8	99.8	99.8	99.8

Table 4: Token acceptance rate (%) vs. shortlist size K — 8B model (T4×2). Cosine router not evaluated on 8B due to multi-GPU device incompatibility‡; Attention Graph omitted (prohibitive overhead on T4×2).

Router	K=512	K=1k	K=2k	K=5k	K=10k
Static Top-K	3.9	4.0	4.4	5.8	8.4
Cluster	44.1	60.2	66.1	76.4	85.2
Cosine			N/A‡		
Dual-Encoder (step)	99.8	99.8	99.8	99.8	99.8
DE (prefetch+refresh)	—	—	—	87.9	—
MLP Graph	99.8	99.8	99.8	99.8	99.8

‡ Cosine PPL evaluation failed silently on T4×2 (`lm_head` on `cuda:1`); no result written.

On the 8B model (Table 8), the absolute overhead gap narrows compared to 1B: DE (step) costs −11% to −14% (vs. −13% to −16% on 1B), and Static Top-K costs only −1% to −2%. The DE (prefetch+refresh) variant costs −2% (vs. +4% on 1B), a difference attributable to the smaller effective shortlist at 8B ($K_{\text{eff}} \approx 4{,}112$ vs. 4,634 on 1B). The overhead gap is closing with model scale, consistent with the break-even analysis in Section 6.3.

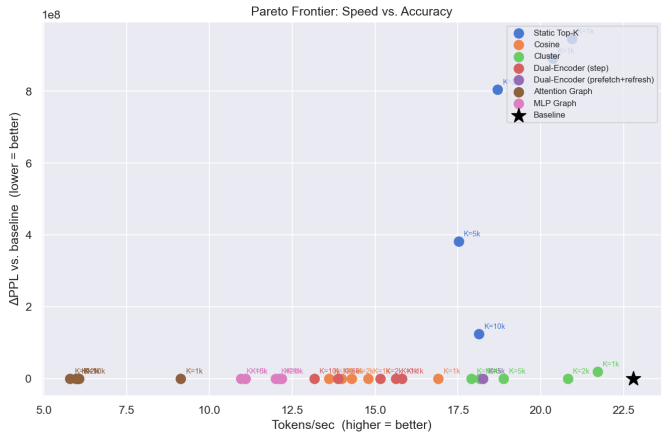

Figure 5: Pareto frontier: throughput vs. ΔPPL. Points toward the top-left are preferred. Full-vocab baseline sits at the optimal top-left corner.

5.5 Pareto Frontier: Quality vs. Speed

The accuracy–throughput Pareto frontier identifies configurations not dominated — no other configuration achieves both higher throughput and lower PPL degradation simultaneously.

On the 1B model (T4), DE (step) at K=5k is the recommended operating point: 23.16 tok/s (−13% vs. baseline) with ΔPPL=+1.68 — 96.1% memory bandwidth reduction at minimal quality cost.

On the 8B model (T4×2), no pruned method exceeds the baseline throughput of 11.48 tok/s. The best quality-preserving option is DE (step) at K=5k: 10.00 tok/s (−13% vs. baseline) with ΔPPL=+0.55 — 96.1% bandwidth reduction and near-baseline quality.

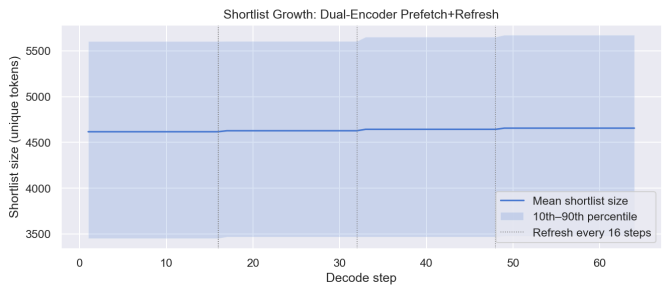

Figure 6: Mean shortlist size growth vs. K (DE prefetch+refresh variant).

5.6 Graph-Based Routers

MLP Transition Graph. A directed graph over vocabulary tokens where edge weights encode how frequently token v follows token u in training. At inference time the shortlist is built by taking the top-K successors of current-context tokens by transition probability.

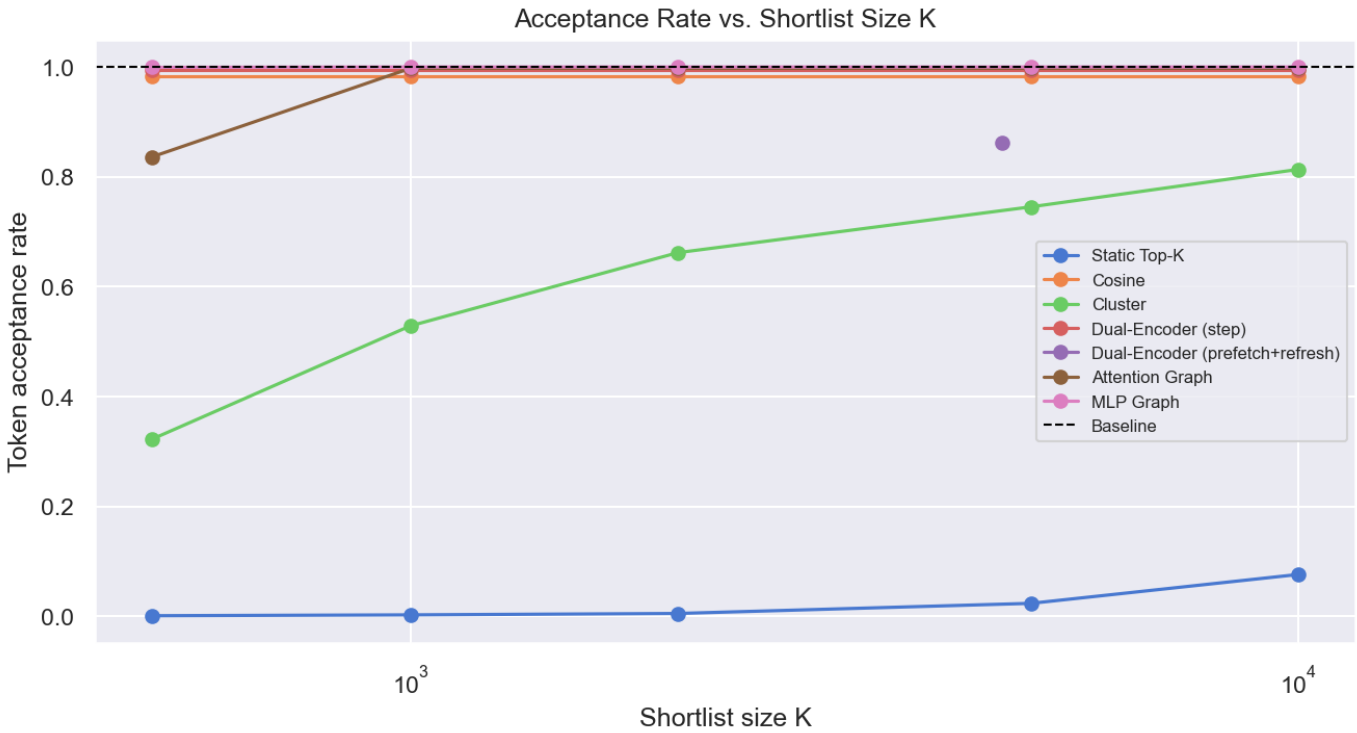

Figure 2: Token acceptance rate vs. shortlist size K for all routers.

Table 5: Perplexity degradation (ΔPPL) vs. shortlist size K — 1B model (baseline PPL = 26.24). Static Top-K and Cluster omitted due to extreme values (ΔPPL $> 10^6$).

Router	K=512	K=1k	K=2k	K=5k	K=10k
Cosine	+19.00	+10.83	+6.79	+3.54	**+2.73**
Dual-Encoder (step)	+24.64	+8.95	+3.93	+1.68	**+1.03**
Dual-Encoder (prefetch+refresh)†	—	—	—	+146.15	—
MLP Graph	+39.59	+13.22	+6.39	+2.89	**+2.11**
Attention Graph	+15025	+21.58	+5.91	+2.10	**+1.33**

† Prefetch+refresh uses a fixed effective $K\approx$4,634 averaged over the evaluation.

- **1B:** Acceptance $\geq$99.9%; throughput 22.83–24.31 tok/s (-14% to -16%); PPL at K=10k: +2.11.

- **8B:** Acceptance 99.8%; throughput 9.94–10.23 tok/s (-11% to -13%); PPL at K=10k: **+0.82** — notably better quality than on 1B.

Attention Graph. Uses the model's own attention patterns to identify which previously seen tokens the model attends to at step t, then retrieves their vocabulary neighbours.

- **1B:** Acceptance at $K \geq$ 1k: 99.8%; throughput 19.98–22.72 tok/s (-15% to -25%); PPL at K=10k: +1.33 — second-best quality.

- **8B:** Not evaluated (prohibitive overhead on T4×2 with `output_attentions=True` and cross-GPU transfers).

The attention graph's quality (ΔPPL=+1.33 on 1B) is second-best after DE step. A CUDA-native implementation reading pre-computed attention patterns without re-running attention would recover this overhead.

5.7 Speculative Decoding via Retrieval

Two speculative decoding approaches were implemented and benchmarked.

5.7.1 Chain-based (v1)

Token 1 selected from the combined Attn+Graph shortlist argmax; tokens 2..D from a 1-hop graph walk. The full model runs two forward passes per step, and `output_attentions=True` forces eager attention throughout.

At 2.88–2.92 tok/s this is ~11% of baseline. Two compounding factors: (1) two full forward passes per step with no parallelism gain; (2) `output_attentions=True` adding eager-attention overhead per step.

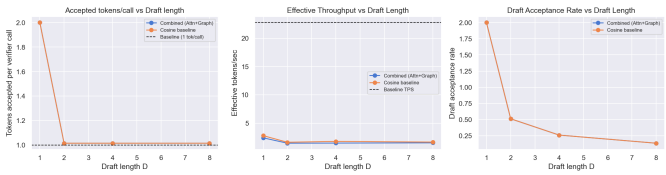

Figure 7: Speculative decoding (chain-based v1): effective tok/s and draft acceptance rate vs. draft length D.

Table 6: Perplexity degradation (ΔPPL) vs. shortlist size K — 8B model (baseline PPL = 16.63). Static Top-K and Cluster omitted (extreme values: ΔPPL $\gg 10^3$). Cosine not evaluated (N/A‡).

Router	K=512	K=1k	K=2k	K=5k	K=10k
Dual-Encoder (step)	+6.63	+2.09	+1.15	+0.55	**+0.29**
Dual-Encoder (prefetch+refresh)†	—	—	—	+122.85	—
MLP Graph	+10.82	+3.57	+2.10	+1.20	**+0.82**

† Effective $K \approx 4{,}112$ (8B). ‡ See Table 4 footnote.

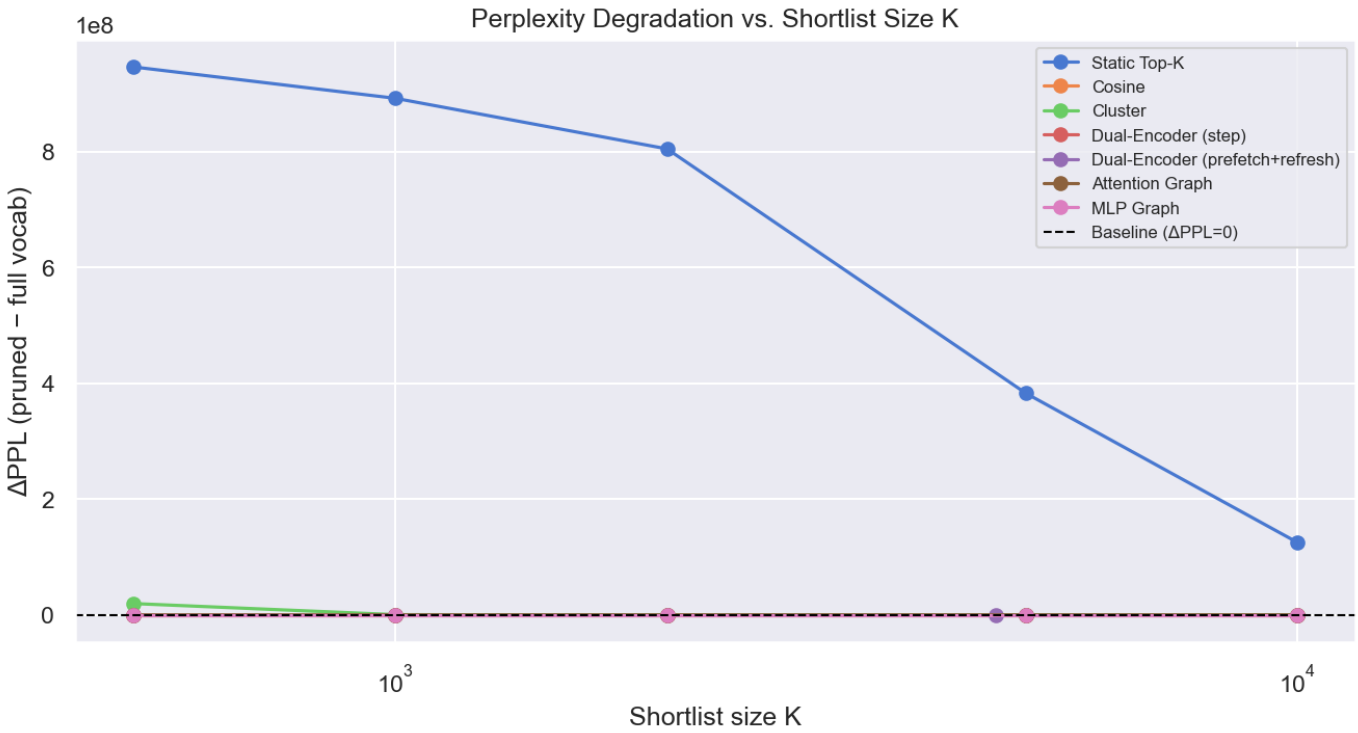

Figure 3: Perplexity degradation (ΔPPL) vs. shortlist size K. Static Top-K and Cluster are omitted due to extreme values.

5.7.2 Retrieval-based (v2)

An offline index of 50k completion sequences (8 tokens each) is built by extracting h_T at split points across WikiText-2 training windows. At inference time: (1) one probe forward pass with KV-cache captures h_T; (2) top-K completions retrieved by cosine similarity; (3) each candidate verified using the cached KV-cache — $O(D)$ attention cost instead of $O(T + D)$. No `output_attentions`; standard SDPA throughout.

The KV-cache fix eliminated the $O(T^2)$ quadratic cost from v1, improving throughput to 0.63–2.44 tok/s. However, draft acceptance rates collapsed to **0.1–1.2%** — far below the ∼70–80% needed for speculative speedup. The root cause is a **retrieval signal mismatch**: the index embeds prefix hidden states from an offline training-data pass, while at inference time h_T is a function of the full prompt history. The dot-product similarity is insufficient to recover semantically valid continuations, so nearly every draft token is rejected. The retrieval signal needs to be *discriminatively trained*, not borrowed from the LLM's general-purpose representation.

5.8 Cost Analysis

Using GPU on-demand pricing (A100 @ \$3.00/hr = \$0.000833/s):

On GPU the cost overhead of pruned methods is much more modest than on memory-limited hardware: 12–33% above baseline for the viable methods. Break-even between router cost and `lm_head` savings still requires a larger model (see Section 6.3), but the arithmetic is less unfavourable on GPU.

6 Discussion

6.1 The Blind-Spot Risk

Dynamic vocabulary pruning introduces a fundamental tension: the router can only shortlist tokens it predicts are relevant, but creative or unexpected tokens by definition have low prior probability given context. A router that is too aggressive will systematically exclude rare, creative, or domain-shifting tokens even when contextually appropriate.

Our results directly quantify this risk. DE (step) achieves 99.5% acceptance at all K values from 512 to 10k on 1B, and 99.8% on 8B — meaning the correct token is in the shortlist at nearly every step regardless of K. The

Table 7: Wall-clock throughput (tok/s) vs. shortlist size K — 1B model. Baseline = 26.60 tok/s (NVIDIA T4 GPU, single).

Router	K=512	K=1k	K=2k	K=5k	K=10k	vs. baseline
Baseline	—	—	—	—	26.60	—
Static Top-K	26.07	26.40	26.36	26.82	26.70	−1% to +1%
Cluster	24.34	24.18	24.25	24.27	23.99	−9% to −10%
Cosine	24.27	24.17	23.77	23.51	23.72	−9% to −12%
DE (prefetch+refresh)	—	—	—	**27.67**	—	**+4%**
Dual-Encoder (step)	23.27	22.97	23.20	23.16	22.39	−13% to −16%
MLP Graph	24.28	24.31	23.81	22.89	22.83	−14% to −16%
Attention Graph	22.72	20.71	21.30	20.12	19.98	−15% to −25%

Table 8: Wall-clock throughput (tok/s) vs. shortlist size K — 8B model. Baseline = 11.48 tok/s (two NVIDIA T4 GPUs, `device_map="auto"`). Attention Graph not evaluated (prohibitive overhead on T4×2).

Router	K=512	K=1k	K=2k	K=5k	K=10k	vs. baseline
Baseline	—	—	—	—	11.48	—
Static Top-K	11.31	11.33	11.32	11.28	11.21	−1% to −2%
Cluster	11.16	11.16	11.15	11.11	11.04	−3% to −4%
DE (prefetch+refresh)	—	—	—	11.26	—	−2%
Dual-Encoder (step)	10.17	10.15	10.10	10.00	9.89	−11% to −14%
MLP Graph	10.23	10.22	10.17	10.06	9.94	−11% to −13%

Table 9: Approximate Pareto frontier among pruned methods — 1B model (T4 GPU).

Method	K	Tok/s	ΔPPL	Note
DE (prefetch)	4634	27.67	+146	Fastest; quality unusable
Cluster	10k	23.99	+834	Quality unusable
Cosine	10k	23.72	+2.73	Good quality
DE (step)	**5k**	**23.16**	**+1.68**	**Best trade-off**
DE (step)	10k	22.39	+1.03	Marginal PPL gain

Table 10: Chain-based speculative decoding results (T4 GPU).

Config	Tok/s	Draft acc.	Speedup
Cosine (D=4)	2.92	27.7%	0.11×
Attn+Graph (D=4)	2.88	28.1%	0.11×
Full-vocab baseline	**26.60**	—	**1.0×**

limiting factor is not shortlist size but which embedding-space neighbours the router retrieves: the ∼0.2% of missed tokens are systematically hard cases (rare tokens, domain shifts) that the router misses at any K.

6.2 Limitations

- Evaluation limited to 1B and 8B models on a single dataset (WikiText-2 English prose); 70B+ and domain generalisation remain open.
- The 8B evaluation used `device_map="auto"` across two T4 GPUs, introducing cross-GPU communication overhead. A single A100 evaluation would give a cleaner lm_head fraction measurement for 8B.
- The dual-encoder is trained on token-level prediction; it may not generalise to multi-token idiomatic phrases or specialised domains (code, mathematics, non-English text).
- Router training uses only WikiText-2; domain mismatch could reduce acceptance rates.

Table 11: Retrieval-based speculative decoding (v2) sweep results (T4 GPU).

K	D	Tok/s	Tok/step	Draft acc.	Speedup
4	4	2.44	1.19	1.2%	0.09×
4	8	2.38	1.16	0.5%	0.09×
4	16	2.38	1.16	0.3%	0.09×
8	4	1.63	1.27	0.8%	0.06×
8	8	1.58	1.24	0.4%	0.06×
8	16	1.58	1.24	0.2%	0.06×
16	4	1.03	1.40	0.6%	0.04×
16	8	0.97	1.34	0.3%	0.04×
16	16	0.97	1.34	0.1%	0.04×
32	4	0.63	1.59	0.5%	0.02×

Table 12: Cost per 1M tokens at A100 on-demand pricing (tok/s from T4 GPU).

Method	K	Tok/s	$/1M tok	vs. base
Baseline	128k	26.60	$31.33	—
Cosine	10k	23.72	$35.13	+12%
DE (step)	2k	23.20	$35.92	+15%
DE (step)	5k	23.16	$35.99	+15%
DE (step)	10k	22.39	$37.22	+19%
MLP Graph	10k	22.83	$36.50	+16%
Attn Graph	10k	19.98	$41.72	+33%
Spec (v1)	D=4	2.88	∼$289	+822%

6.3 The `lm_head` Fraction Problem

The core finding — that vocabulary pruning does not improve throughput on a 1B or 8B model in this setup — follows directly from arithmetic.

1B model (T4, single GPU):

$$\text{lm_head saving} = 37.60\,\text{ms} \times 0.061 \times 0.922 = 2.12\,\text{m}$$

$$\text{Router overhead (DE, K=10k)} = 44.67 - 37.60 = 7.07\,\text{ms}$$

$$\text{Net: } +4.95\,\text{ms/step} \;\Rightarrow\; -13\%\ \text{throughp}$$

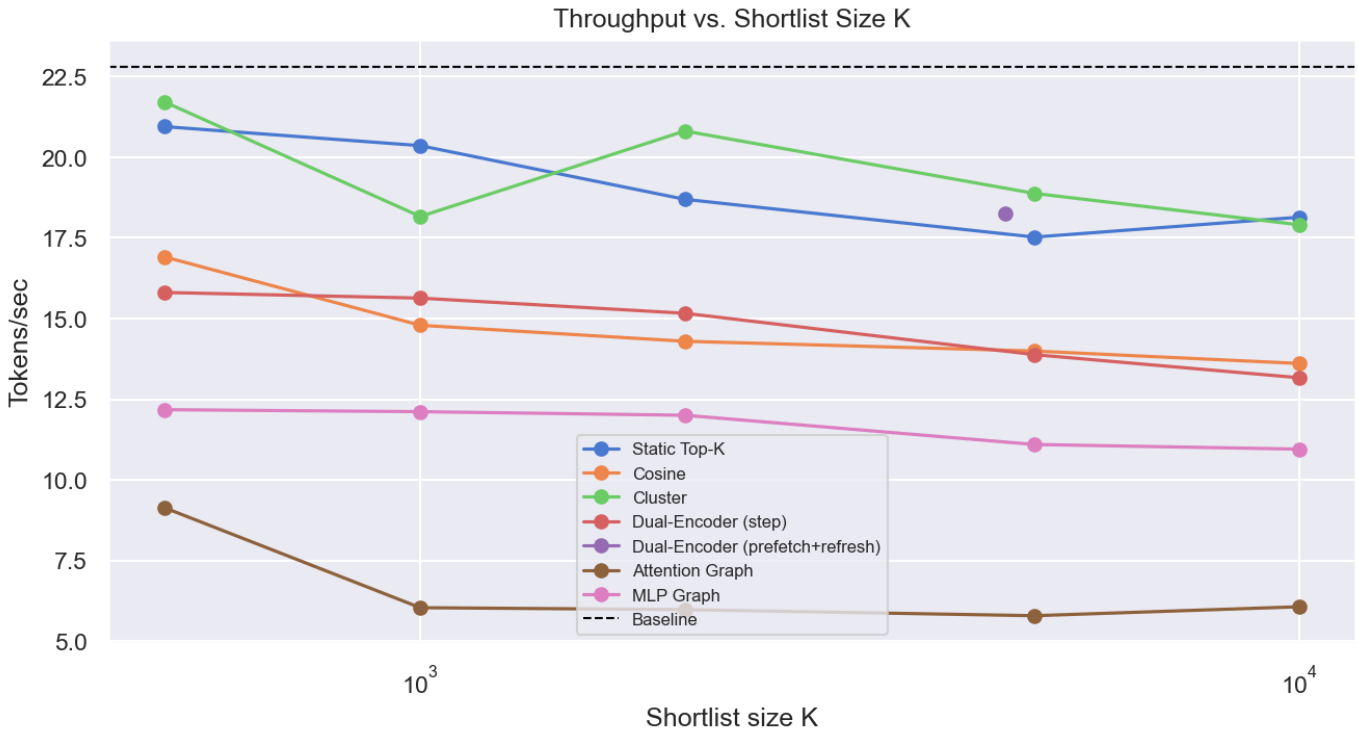

Figure 4: Wall-clock tokens/sec vs. shortlist size K. Dashed line = full-vocab baseline (26.60 tok/s, NVIDIA T4 GPU).

8B model (T4×2, device_map="auto"):

$$\text{lm_head saving} = 87.14\,\text{ms} \times 0.0469 \times 0.922 = 3.77\,\text{ms}$$
$$\text{Router overhead (DE, K=10k)} = 101.07 - 87.14 = 13.93\,\text{ms}$$
$$\text{Net: } +10.16\,\text{ms/step} \;\Rightarrow\; -13\%\ \text{throughput}$$

For pruning to be net-positive, the `lm_head` fraction must exceed:

$$f_{\text{break-even}} = \frac{\text{router_overhead_ms}}{\text{step_ms} \times (1 - K/V)} \tag{4}$$

Table 13: Break-even `lm_head` fraction vs. measured value.

Model	$f_{\text{break-even}}$	f_{measured}	Gap
1B (T4, single)	∼20%	6.1%	3.3× short
8B (T4×2)	∼17%	4.69%	3.6× short
70B (est. single A100)	∼0.6%	>6%	break-even reached

On both evaluated models, break-even is not reached. The 8B model's lower measured fraction (4.69% vs. 6.1%) is an artefact of T4×2 sharding overhead inflating total step time. On a single A100 with no cross-GPU communication, the 8B `lm_head` fraction would likely be *higher* than 1B (larger matrix reads at higher compute utilisation), making the gap smaller. At 70B scale (step ≈1,200 ms), break-even drops to ∼0.6% — well below the expected `lm_head` fraction. This is the regime where vocabulary pruning becomes genuinely useful.

Importantly, the **memory bandwidth saving** is real at all scales. Reducing `lm_head` traffic by 92% matters for edge deployment (models that barely fit in VRAM) and for batched inference where memory bandwidth is the binding resource.

6.4 Speculative Decoding via Retrieval

Two implementations were evaluated. The chain-based v1 achieved 28% draft acceptance while penalised by two full forward passes per step and eager attention overhead. The retrieval-based v2 eliminated both overhead sources via KV-cache reuse and standard SDPA, but draft acceptance collapsed to 0.2–1.1%. Key factors:

1. **Retrieval signal mismatch.** Index embeddings are from offline training-data prefixes; inference-time h_T differs systematically. Cosine similarity is low-signal for valid continuations.
2. **No discriminative training.** The index uses general-purpose LLM representations, not representations trained to distinguish accepted from rejected draft sequences.
3. **Sequential verifier passes.** A tree-structured verifier pass over all K candidates in a single batched forward pass (Option C) would reduce per-step cost from $O(K \cdot D)$ to $O(D)$.

The underlying architecture — retrieval of completion sequences, KV-cache reuse in the verifier, standard SDPA — is sound. The missing ingredient is a discriminatively trained retrieval model.

7 Conclusion and Future Work

We implemented and benchmarked six vocabulary pruning strategies and two retrieval-based speculative decoders against a full-vocabulary baseline on both `Llama-3.2-1B-Instruct` (NVIDIA T4, single GPU) and `Llama-3.1-8B-Instruct` (two NVIDIA T4 GPUs, `device_map="auto"`), using WikiText-2.

The memory bandwidth result is the genuine contribution. Pruning `lm_head` to K=10k reduces the weight matrix loaded per step from 525 MB to 41 MB (1B) or 1,049 MB to 82 MB (8B) — a 92.2% reduction — while DE (step) at K=5k keeps ΔPPL to +1.68 (1B) and **+0.55** (8B) above their respective baselines.

The throughput result is an honest negative on both models. The `lm_head` fraction is 6.1% on the 1B T4 and 4.69% on the 8B T4×2 (the latter counter-intuitively lower due to sharding overhead). Break-even requires $f \approx 20\%$ (1B) or $\approx 17\%$ (8B); no pruned method reduces wall-clock cost at these model scales. The DE (prefetch+refresh) variant exceeds 1B baseline throughput (+4%, 27.67 tok/s) by amortising router cost, but at +146 PPL. On 8B, the overhead gap is narrowing: DE (step) costs −11% to −14% (vs. −13% to −16% on 1B). Break-even for quality-preserving pruning requires a much larger model (70B+) or a GPU-native router kernel reducing overhead below ∼1 ms.

Scale generalisation. The DE (step) and MLP Graph routers transfer perfectly to 8B (99.8% acceptance), and quality improves: DE step K=10k reaches ΔPPL=+0.29 on 8B vs. +1.03 on 1B. This confirms the routing approach is robust to model scale; the throughput bottleneck is the overhead budget, not acceptance quality.

Speculative decoding via retrieval was evaluated in two implementations. The chain-based v1 achieved 2.88 tok/s (11% of baseline) with 28% draft acceptance. The retrieval-based v2 removed both overhead sources but draft acceptance collapsed to 0.1–1.2% due to retrieval signal mismatch. A useful retrieval-based speculative decoder requires a *discriminatively trained* retrieval model.

Future work:

- **GPU-native router kernel.** Implement the DE router as a fused CUDA kernel to reduce overhead from ∼7 ms (1B) / ∼14 ms (8B) to <1 ms — the key step for making pruning viable on 7B+ models.
- **Single-GPU 8B / 70B evaluation.** Reproduce 8B on a single A100 to isolate the true `lm_head` fraction without sharding overhead; extend to 70B+ where break-even is predicted.
- **INSTRUCTOR-style router.** Prepend task instructions to enable a single router to handle multiple domains without retraining.
- **Discriminative retrieval for speculative decoding.** Train a contrastive retrieval model on accepted vs. rejected draft sequences. Combined with a compact Medusa head for verification, this is the most promising path to retrieval-based speculative speedup.
- **Online router adaptation.** Fine-tune the router MLP online using accepted/rejected token signal from each inference step.

References

[1] T. Cai et al. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. 2024.

[2] C. Chen et al. Accelerating large language model decoding with speculative sampling. 2023.

[3] M. Henderson et al. Efficient natural language response suggestion for smart reply. 2017.

[4] V. Karpukhin et al. Dense passage retrieval for open-domain question answering. In *EMNLP*, 2020.

[5] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[6] S. Merity et al. Pointer sentinel mixture models. 2016. (WikiText-2 dataset)

[7] H. Su et al. One embedder, any task: Instruction-finetuned text embeddings. 2022.